Aula 9 — Análise Sintática: Visão Geral
Descendente x Ascendente

Compiladores I

FACOM - Faculdade de Computação

Bloco 4: Análise Sintática

Duração estimada: 70–80 minutos

Revisão do bloco de Análise Léxica

- O scanner transforma código-fonte em uma sequência de tokens;
- Encerramos com a implementação prática de um scanner.

Pergunta rápida

O que vem depois da lista de tokens no pipeline de compilação?

Objetivos de aprendizagem

- Explicar o papel do analisador sintático na verificação da estrutura do programa;
- Diferenciar as estratégias de análise descendente e ascendente;
- Relacionar cada estratégia a classes de gramáticas apropriadas.

Sumário da aula

O papel do analisador sintático

Estratégias de análise

Da lista de tokens à árvore sintática

Entrada / Saída

Entrada: sequência de tokens (Aulas 5–8).
Saída: uma **árvore sintática** (ou Árvore Sintática Abstrata, ASA), representando a estrutura hierárquica do programa segundo a gramática da linguagem (Aula 3).

Exemplo — No Micro C

O Capítulo 3 do livro implementa o parser que lê os tokens do arquivo `tokens.txt` (gerado pela fase léxica) e monta a ASA.

O que a análise sintática garante — e o que não garante

- Garante que a **sequência** de tokens obedece à gramática (ex: todo `if` tem uma condição entre parênteses);
- **Não** garante coerência de significado (ex: usar uma variável não declarada) — isso é tarefa da análise semântica (próximo bloco);

Pergunta rápida

`int x = y + 1;` (sendo `y` nunca declarado) é sintaticamente válido? Justifique.

Sumário da aula

O papel do analisador sintático

Estratégias de análise

Análise descendente (top-down)

Ideia central

Constrói a árvore sintática a partir da **raiz**, tentando prever qual produção aplicar com base no(s) próximo(s) token(s), expandindo até chegar aos tokens da entrada.

- Intuitiva de implementar manualmente (descida recursiva — Aula 11);
- Exige gramáticas em uma forma específica (sem recursão à esquerda, por exemplo).

Análise ascendente (bottom-up)

Ideia central

Constrói a árvore sintática a partir das **folhas** (tokens), combinando-as progressivamente (reduções) até alcançar o símbolo inicial da gramática.

- Mais poderosa: aceita uma classe maior de gramáticas (ex: com recursão à esquerda);
- Geralmente implementada com auxílio de geradores de parser (ex: Yacc/Bison), pois é mais complexa de programar manualmente.

Comparando as duas estratégias

	Descendente	**Ascendente**
Construção da árvore	raiz $\rightarrow$ folhas	folhas $\rightarrow$ raiz
Implementação manual	mais simples	mais complexa
Classe de gramáticas	mais restrita (LL)	mais ampla (LR)
Uso típico	parsers manuais	geradores de parser

Pergunta rápida

Por que a análise descendente exige gramáticas sem recursão à esquerda, mas a ascendente não tem essa restrição?

Desafio em sala

Para a sentença `a + b * c`, esboce (em alto nível, sem se preocupar com o algoritmo exato) a ordem em que uma análise descendente e uma análise ascendente construiriam a árvore sintática: qual constrói primeiro o nó da raiz? Qual constrói primeiro os nós-folha?

Síntese da aula

- A análise sintática verifica a *estrutura* do programa e produz a árvore sintática;
- Análise descendente: raiz $\rightarrow$ folhas, mais simples de implementar manualmente;
- Análise ascendente: folhas $\rightarrow$ raiz, mais poderosa, geralmente via ferramentas.

Próxima aula: analisadores sintáticos preditivos — conjuntos FIRST e FOLLOW.

Teste rápido — Aula 9

Questões teóricas

1. Qual é a entrada e a saída da análise sintática?
2. Compare, em uma frase cada, análise descendente e análise ascendente.

Questão prática

3. O programa `int x = ;` tem um erro sintático. Explique qual regra da gramática de declaração é violada.